1. What is a Stack?

Answer:

A Stack is a linear data structure that follows the **LIFO (Last In, First Out)** principle, where the last inserted element is the first one to be removed.

2. What does LIFO stand for?

Answer:

LIFO stands for **Last In, First Out**.

3. Why is a Stack used in an Undo/Redo system?

Answer:

A Stack stores the sequence of actions. The most recent action can be undone first, which follows the LIFO principle.

4. How many stacks are required for an Undo/Redo system?

Answer:

Two stacks are required:

- **Undo Stack**
- **Redo Stack**

5. What happens when a new change is made?

Answer:

The new change is pushed onto the **Undo Stack**, and the **Redo Stack** is cleared.

6. What happens during an Undo operation?

Answer:

The latest action is popped from the **Undo Stack** and pushed onto the **Redo Stack**.

7. What happens during a Redo operation?

Answer:

The latest action is popped from the **Redo Stack** and pushed back onto the **Undo Stack**.

8. Which Stack operation is used to insert an element?

Answer:

Push operation.

9. Which Stack operation is used to remove an element?

Answer:

Pop operation.

10. What is the Peek operation?

Answer:

Peek returns the top element of the stack without removing it.

11. What is Stack Overflow?

Answer:

Stack Overflow occurs when an attempt is made to push an element into a full stack.

12. What is Stack Underflow?

Answer:

Stack Underflow occurs when attempting to pop an element from an empty stack.

13. What is the time complexity of the Push operation?

Answer:

O(1)

14. What is the time complexity of the Pop operation?

Answer:

O(1)

15. What is the time complexity of the Peek operation?

Answer:

O(1)

16. What is the space complexity of the Undo/Redo system?

Answer:

O(n), where **n** is the number of stored actions.

17. Why is the Redo Stack cleared after making a new change?

Answer:

Because the previous undone actions are no longer valid after a new modification is made.

18. Which Python data structure is commonly used to implement a Stack?

Answer:

A **list**.

19. Which Python methods are used for Stack implementation?

Answer:

- append() → Push
- pop() → Pop

20. Can a Stack be implemented using other data structures?

Answer:

Yes. A Stack can be implemented using:

- Lists
- Linked Lists
- Collections (deque)

21. Why is Stack preferred over Queue for Undo/Redo?

Answer:

Undo requires reversing the most recent action first, which follows the **LIFO** order. A Queue follows **FIFO**, making it unsuitable.

22. What is FIFO?

Answer:

FIFO stands for **First In, First Out**, which is the principle followed by a Queue.

23. Give real-life applications of a Stack.

Answer:

- Undo/Redo in text editors
- Browser Back button
- Function call management

- Expression evaluation
- Parentheses matching

24. What happens if the Undo Stack becomes empty?

Answer:

No more actions can be undone.

25. What happens if the Redo Stack becomes empty?

Answer:

No more actions can be redone.

26. What is the current document state after an Undo operation?

Answer:

It reflects the document after removing the most recent change.

27. How is the document state displayed?

Answer:

By showing the current content after applying all valid changes stored in the Undo Stack.

28. What is the advantage of using two stacks instead of one?

Answer:

One stack stores performed actions (Undo Stack), while the other stores undone actions (Redo Stack), allowing efficient undo and redo operations.

29. What are the advantages of using a Stack?

Answer:

- Simple implementation
- Fast insertion and deletion (**O(1)**)
- Efficient for tracking recent actions
- Ideal for Undo/Redo functionality

30. What are the applications of an Undo/Redo system?

Answer:

- Text editors (e.g., Microsoft Word)

- Code editors (e.g., Visual Studio Code)
- Graphic design software
- Image editing applications
- CAD software

Frequently Asked Viva Question

Q. Why is Stack the best data structure for implementing an Undo/Redo system?

Answer:

A Stack follows the **Last In, First Out (LIFO)** principle, meaning the most recent change is undone first. This matches the natural behavior of Undo/Redo operations, making Stack the most efficient and appropriate data structure for such applications.